



Universidad
Tecnológica
del Perú

PRIMERA UNIDAD DE APRENDIZAJE 3: Patrones estructurales

CURSO: DISEÑO DE PATRONES

SESIÓN N° 08:

**Patrones Estructurales:
Introducción a patrones estructurales.
Patrón Adapter. Patrón Facade.**

DOCENTES DE UTP
SISTEMAS



Lo que debemos tener en cuenta



5 minutos



¿QUÉ SON PATRONES Adapter y Facade?

Facade Pattern:

Facade: Proporciona una interfaz simplificada para un conjunto de interfaces en un subsistema. El Facade delega las llamadas de los clientes a los objetos del subsistema.

SubsystemClass1, SubsystemClass2, SubsystemClass3: Estas son clases que representan diferentes partes del subsistema. Cada una tiene operaciones específicas que el Facade puede utilizar.

Adapter Pattern:

Adapter: Adapta la interfaz de una clase (Adaptee) a otra interfaz esperada por el cliente (Target).

Target: Define la interfaz específica que el cliente usa.

Adaptee: Tiene una interfaz incompatible con la esperada por el cliente.

Client: Interactúa con el sistema a través de la interfaz Target y no necesita conocer la existencia del Adaptee.



¿Qué vimos en la sesión anterior?



En el Patrón de diseño de clase Factory, el método principal que crea objetos se llama _____. El Patrón Factory promueve el principio de diseño _____, que sugiere que se debe depender de abstracciones y no de concreciones. Una ventaja del Patrón Factory es que facilita la creación de _____ sin especificar la clase exacta del objeto que se creará. El Patrón Factory ayuda a reducir el _____ del código al centralizar la lógica de creación de objetos. En el Patrón Factory, las subclases pueden alterar el tipo de objetos que serán creados mediante la sobreescritura del método _____.

☐ Show answers

objetos

factoryMethod

acoplamiento

factoryMethod

SOLID



Unidad de aprendizaje 3: Patrones estructurales.	Semana 8,9,10 y 11
Logro específico de aprendizaje: Al finalizar la unidad el participante aplica los patrones estructurales para la solución de problemas.	
Temario: • Patrones Estructurales: Introducción a patrones estructurales. Patrón Adapter. Patrón Facade. • Patrones Estructurales: Patrón Decorator. Patrón Composite. • Consolidación de temas (Repaso). • Patrones Estructurales: Patrón Proxy. Patrón Bridge.	

Al final de la sesión el estudiante aplica patrones estructurales:

Patrón Facade. Patrón Adapter en un contexto definido



Aprender patrones estructurales es vital porque ayudan a simplificar sistemas complejos, mejoran la organización del código y facilitan la reutilización de componentes. Estos patrones promueven el desacoplamiento, haciendo el software más flexible y fácil de mantener. Además, proporcionan soluciones probadas para problemas comunes de diseño, lo que ahorra tiempo y reduce errores. Entender estos patrones también mejora la capacidad para comunicar diseños y colaboraciones efectivamente con otros desarrolladores. En resumen, dominarlos contribuye a la creación de software robusto y escalable.



¡Que no se te escape nada de tu clase!:

SESIÓN N° 08:

**Patrones Estructurales:
Introducción a patrones estructurales.
Patrón Adapter. Patrón Facade**

Los patrones estructurales sirven para resolver problemas de diseño estableciendo relaciones estructurales entre las entidades de maneras que suelen sugerir los nombres metafóricos de los patrones.

Estas relaciones estructurales hacen que los objetos sean más ricos en términos de comportamiento, más poderosos o más fáciles de usar debido a la colaboración entre objetos.

La agregación de objetos es el principal mecanismo relacional utilizado para todos los patrones estructurales, aunque una relación contextual puede caracterizarse de manera más restringida como composición, adaptación, delegación, etc. para reflejar la forma en que los objetos colaboran para brindar una solución a un problema particular donde se utiliza un patrón.

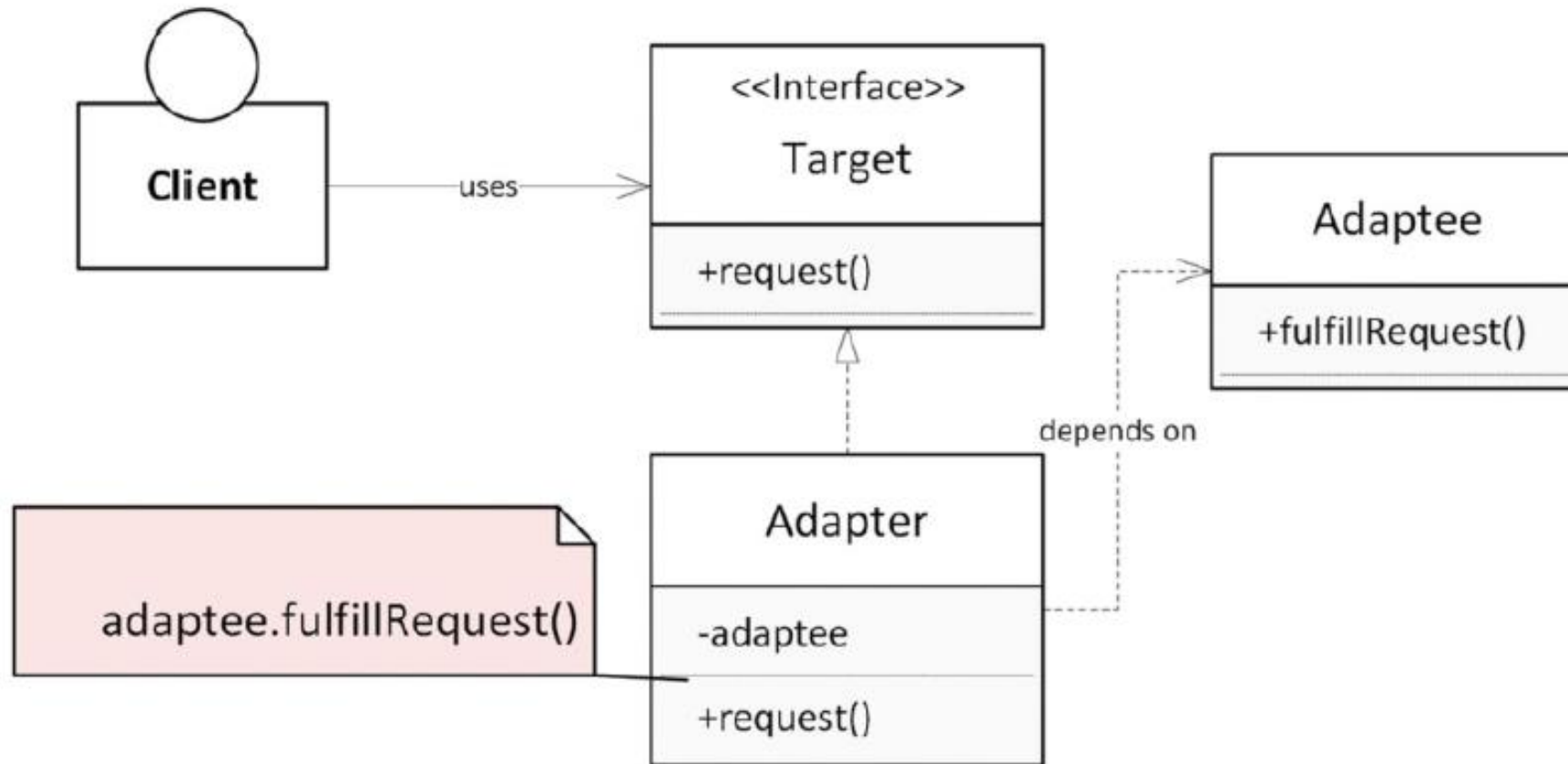
El nombre del patrón parece acertado en lo que respecta a su función: resolver problemas de “desajuste”.

Un desajuste puede ser tan pequeño como un conjunto de parámetros que no coinciden al pasarse a un módulo de destino o tan grande como las comunicaciones entre sistemas con protocolos de comunicación, entornos operativos o transferencias de información que no coinciden.

Los sistemas de software operan cada vez más en la nube, pero aún pueden necesitar depender de los sistemas existentes para obtener funcionalidades.

Por lo tanto, los desajustes de todo tipo deben abordarse en cualquier esfuerzo por migrar operaciones de software a un entorno de nube.

Target es una abstracción en la que la solicitud es el método que el cliente debe utilizar. Adapter implementa Target y utiliza una instancia de Adaptee para invocar, con la adaptación adecuada, una operación que cumpla con la solicitud.

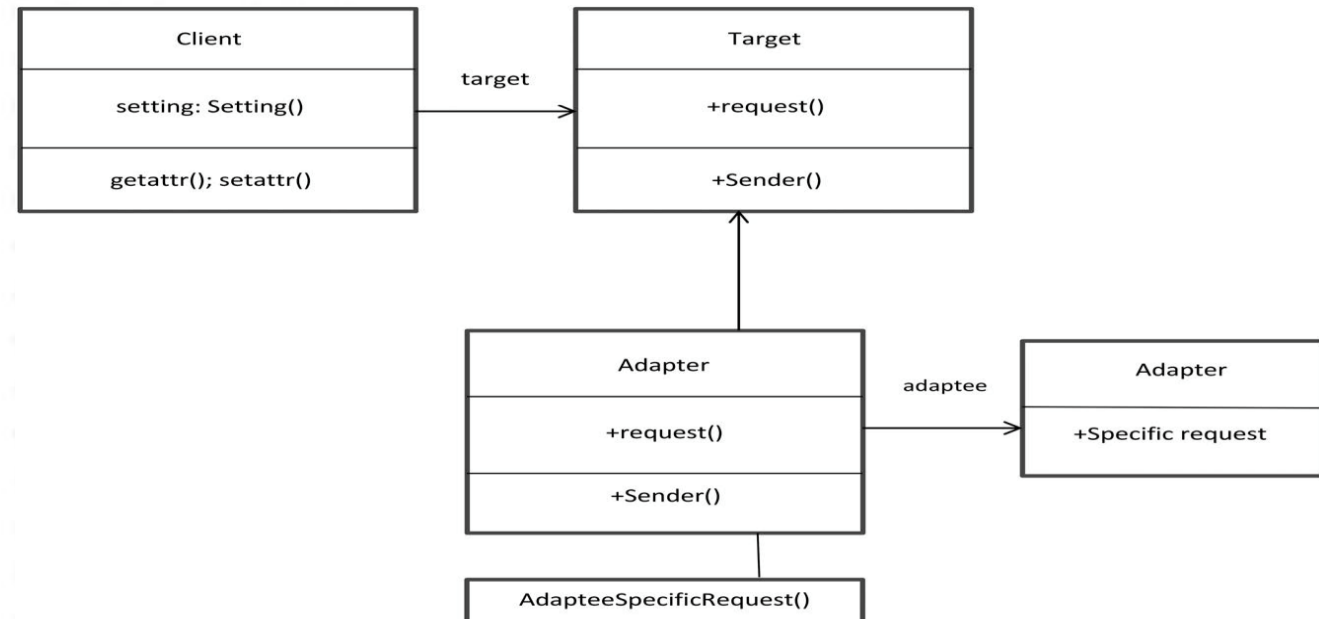


Patrones estructurales – Adapter - ejemplo

Una empresa de comercio electrónico utiliza un sistema antiguo de gestión de inventarios que no es compatible con su nueva aplicación de ventas en línea.

La nueva aplicación requiere datos de inventario en un formato diferente al que proporciona el sistema antiguo.

Para resolver este problema sin modificar el sistema existente, se implementa el patrón Adapter.



```
// Interfaz existente en la nueva aplicación de ventas en línea
interface InventoryService {
    String getInventoryItem(String itemId);
}

// Clase Adaptee del sistema antiguo de gestión de inventarios
class OldInventorySystem {
    public String getItemDetails(String itemId) {
        // Devuelve detalles del artículo en un formato diferente
        return "Item ID: " + itemId + ", Quantity: 100, Location: Warehouse
A";
    }
}
```

Patrones estructurales – Adapter - ejemplo

```
// Clase Adapter que adapta la interfaz InventoryService a OldInventorySystem
class InventoryAdapter implements InventoryService {
    private OldInventorySystem oldInventorySystem;

    public InventoryAdapter(OldInventorySystem oldInventorySystem) {
        this.oldInventorySystem = oldInventorySystem;
    }

    @Override
    public String getInventoryItem(String itemId) {
        // Convierte el formato de salida del sistema antiguo al formato requerido
        String oldDetails = oldInventorySystem.getItemDetails(itemId);
        // Realiza las conversiones necesarias (esto es solo un ejemplo simple)
        String[] details = oldDetails.split(", ");
        return "ID: " + details[0].split(": ")[1] + ", Stock: " + details[1].split(": ")[1];
    }
}
```

```
// Cliente que utiliza la interfaz InventoryService
public class AdapterPatternBusinessDemo {
    public static void main(String[] args) {
        InventoryService inventoryService = new InventoryAdapter(new
        OldInventorySystem());
        String itemDetails = inventoryService.getInventoryItem("12345");
        System.out.println(itemDetails);
    }
}
```

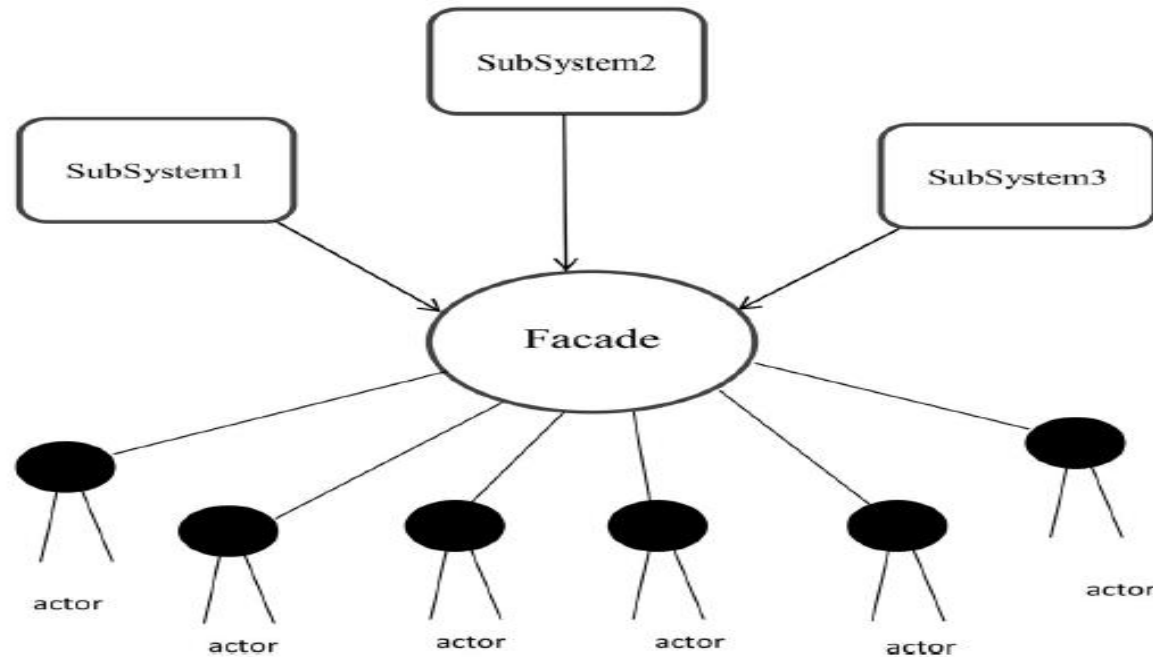
InventoryService: La interfaz que la nueva aplicación de ventas en línea utiliza para obtener información de inventario.

OldInventorySystem: La clase del sistema antiguo de gestión de inventarios que proporciona detalles en un formato diferente.

InventoryAdapter: La clase adaptadora que implementa la interfaz **InventoryService** y traduce las llamadas al **OldInventorySystem**.

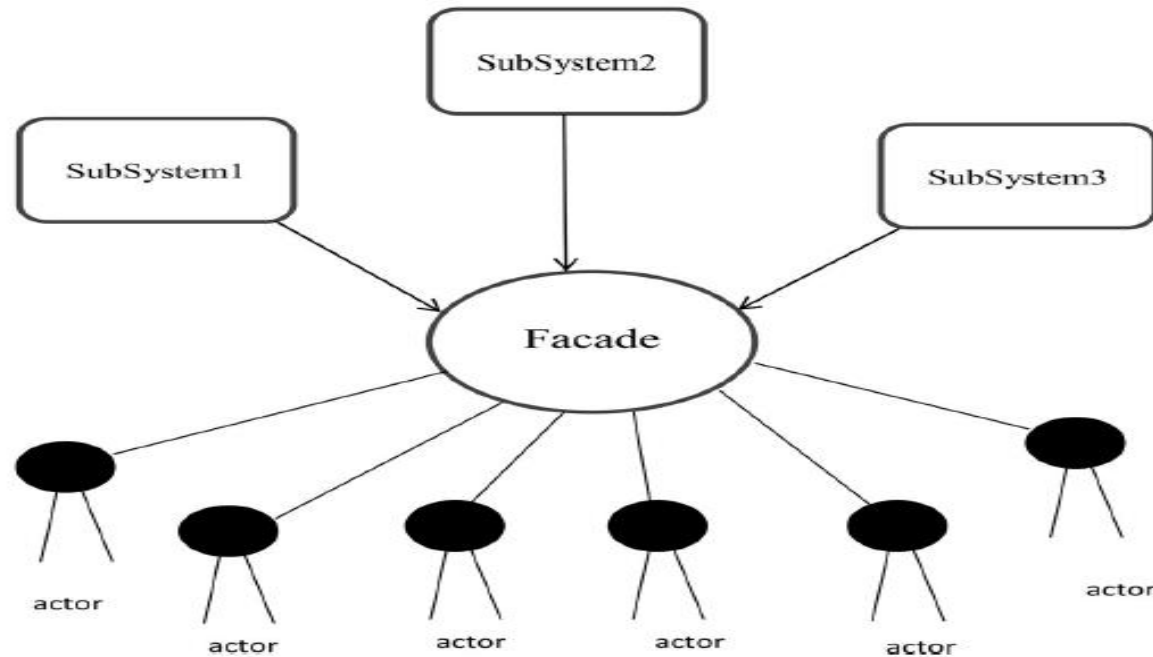
AdapterPatternBusinessDemo: Clase cliente que utiliza la interfaz **InventoryService** para obtener detalles de inventario en el formato requerido por la nueva aplicación de ventas en línea.

El patrón de fachada es un patrón de diseño estructural que crea una interfaz más unificada para un sistema más complejo. El término fachada se refiere a la cara de un edificio o, más específicamente, a la interfaz exterior de un sistema complejo compuesto por varios subsistemas. Es una parte importante de los patrones de diseño de la Banda de los Cuatro. Simplifica el acceso a los métodos de los sistemas subyacentes al proporcionar un único punto de entrada.



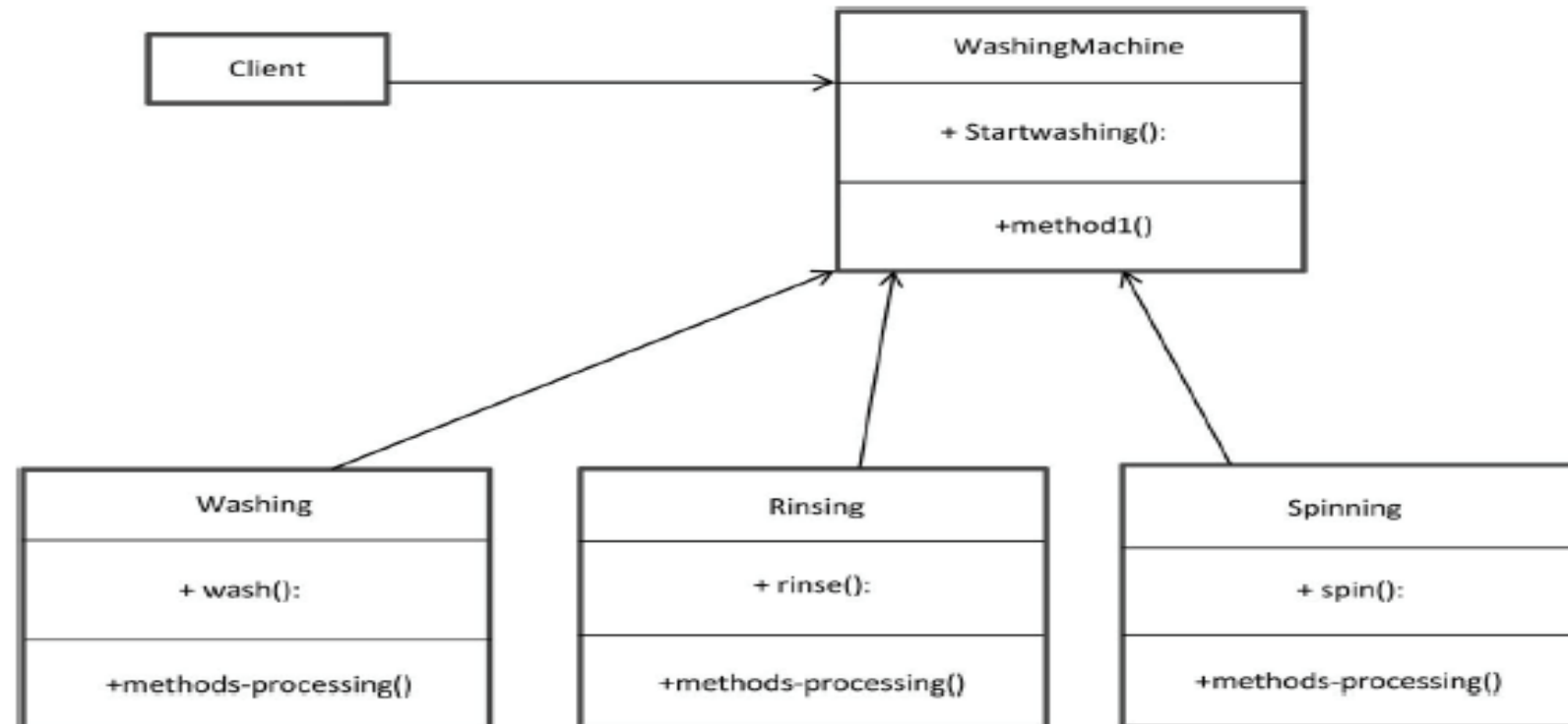
Patrones estructural – Facade - ejemplo

Se utiliza un sistema de gestión de inventario para organizar los artículos. Sin embargo, como el cliente no necesita saber nada sobre el inventario, es preferible que le pida al comerciante una lista de artículos, ya que el comerciante sabe dónde se encuentra cada artículo. El comerciante actúa como interfaz de fachada en este caso.



Patrones estructural – Facade - ejemplo

Supongamos que tenemos una lavadora que puede lavar, enjuagar y centrifugar la ropa, pero que realiza cada tarea por separado. Debemos abstraer las complejidades de los subsistemas porque el sistema en su conjunto es bastante complejo. Necesitamos un sistema que pueda automatizar toda la tarea sin nuestra interferencia.



Patrones estructurales – Facade - ejemplo

```
// Subsystem Class: Centrifugar
class Centrifugado {
    public void centrifugar() {
        System.out.println("Centrifugando la ropa...");
    }
}
```

```
// Facade Class
class LavadoraFacade {
    private Lavado lavado;
    private Enjuague enjuague;
    private Centrifugado centrifugado;

    public LavadoraFacade() {
        this.lavado = new Lavado();
        this.enjuague = new Enjuague();
        this.centrifugado = new Centrifugado();
    }
```

```
    public void lavarRopa() {
```

Patrones estructurales – Facade - ejemplo

```
// Subsystem Class: Lavar
class Lavado {
    public void lavar() {
        System.out.println("Lavando la ropa...");
    }
}
```

```
// Subsystem Class: Enjuagar
class Enjuague {
    public void enjuagar() {
        System.out.println("Enjuagando la ropa...");
    }
}
```

Patrones estructurales – Facade - ejemplo

```
public void lavarRopa() {  
    lavado.lavar();  
}  
public void enjuagarRopa() {  
    enjuague.enjuagar();  
}  
public void centrifugarRopa() {  
    centrifugado.centrifugar();  
}  
}  
// Cliente  
public class FacadePatternDemo {  
    public static void main(String[] args) {  
        LavadoraFacade lavadora = new LavadoraFacade();  
        lavadora.lavarRopa();  
        lavadora.enjuagarRopa();  
        lavadora.centrifugarRopa();  
    }  
}
```

¿Cuál es la diferencia entre Facade y Adapter?



Caso práctico



Realizar ejercicio práctico es una excelente manera de aprender y entender cómo elaborar un patrón de diseño estructural. Aquí tienes un ejercicio práctico detallado que puedes seguir:

Ejercicio Práctico:

Caso de Uso: Plataforma de Pago en Línea

Un hospital necesita integrar múltiples sistemas para gestionar la información del paciente, como registros médicos electrónicos (EMR), sistemas de laboratorio y sistemas de facturación.

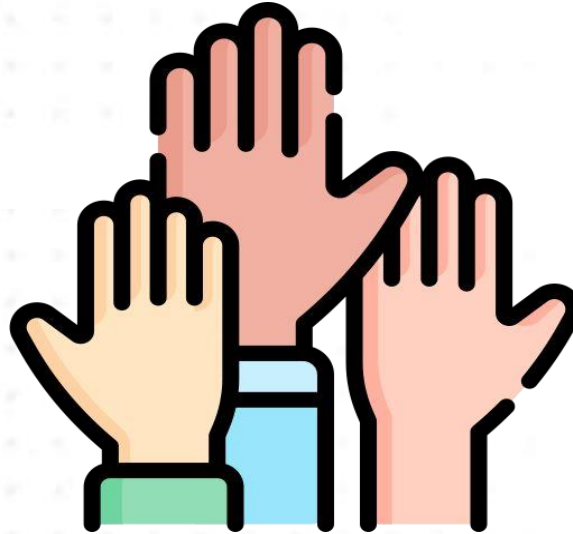
Aplicación del Patrón Facade: Se implementa una fachada que proporciona una interfaz unificada para acceder a estos sistemas dispares. La fachada ofrece métodos simplificados para obtener la historia clínica completa de un paciente, resultados de laboratorio y estado de facturación sin que los usuarios necesiten interactuar directamente con cada sistema.

Objetivos:

- 1.Elaborar el código java y diagrama de clase UML utilizando el patrón seleccionado.**



¿Quién quisiera participar? (2 voluntarios)





¿Qué se les hizo más fácil?
¿Qué se les hizo más retador?





¿Qué hemos aprendido el día de hoy?



Tomar apuntes de manera eficaz ayuda a consolidar el aprendizaje y prepararse para los exámenes.



Facade: Proporciona una interfaz simplificada para un conjunto de interfaces en un subsistema, ocultando la complejidad y ofreciendo una única entrada para interactuar con múltiples componentes.

Adapter: Permite que una interfaz incompatible sea utilizada por otra. Actúa como un puente entre dos interfaces, adaptando la interfaz de un objeto a la que el cliente espera.



- Elabora la actividad práctica de acuerdo a la guía de laboratorio de sesión
- Sube la actividad práctica en la plataforma virtual de aprendizaje
- Guarda la actividad con la siguiente etiqueta:
DPA_Actividad08_NombreApellido

Nota: No olvides también revisar tu plataforma UTP+Class



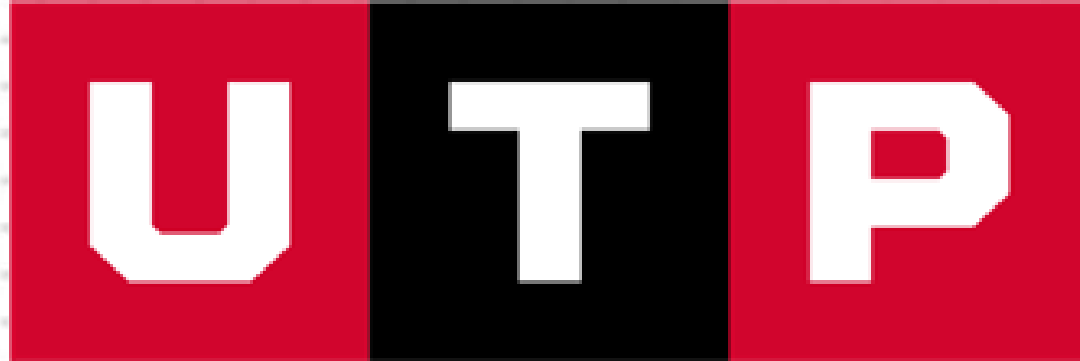
- Hu, C. (2023). *An Introduction to Software Design: Concepts, Principles, Methodologies, and Techniques*. Springer Nature.
- bin Uzayr, S. (2023). *Software Design Patterns: The Ultimate Guide*. CRC Press.

Nota: No olvides también revisar tu plataforma UTP+Class



**MUCHAS GRACIAS QUE
DIOS LOS BENDIGA!!!**





**Universidad
Tecnológica
del Perú**

